

Real-Time Resource Monitoring Framework in a Heterogeneous Kubernetes Cluster

Balqees Talal Hasan
Department of Computer Sciences
University of Mosul
Mosul, Iraq
Balqeeatalal.cs@gmail.com

Dhuha Basheer Abdullah
Department of Computer Sciences
University of Mosul
Mosul, Iraq
prof.dhuha_basheer@uomosul.edu.iq

Abstract— Container-based virtualization is quickly becoming the standard method for developing and deploying applications. Kubernetes is a well-known open-source project that provides a platform for containerized application orchestration. In such a container-based environment, it is critical that Kubernetes can monitor the cluster computing resource as well as the resource requirements of the running pods. The goal of this paper is to introduce a generic framework which is named as Kube-Monitor for real-time monitoring of a heterogeneous Kubernetes cluster's resources, which include traditional data center devices and raspberry pi devices, as well as monitoring pod resource requirements. Kube-Monitor will collect 15 different cluster resource metrics and update their values every five seconds. As a result, the Kubernetes operator can use the Kube-monitor framework to view a comprehensive set of continuously updated resource metrics.

Keywords— Cluster Computing; Docker; Edge Computing
Kubernetes;Monitoring; Raspberry Pi

I. INTRODUCTION

Container-based virtualization has recently emerged as a key technology for deploying applications in cloud computing data centers. Containerization, also known as operating system (OS)-level virtualization, divides a server into multiple virtual instances, known as containers. Containers, unlike traditional virtual machines (VMs), can be started quickly, resulting in lower computing resource usage and deployment time [1].

Docker is a containerization software that was released in 2013 as an open-source project. Docker was created as a reliability layer that separates a program's software dependencies from the physical properties of the hardware. This enables a single Docker application to run on any hardware and operating system combination as long as the combination has the base Docker run time engine[2].

In order to manage container deployments in a large-scale system, an orchestration platform is required. Kubernetes is the most widely used container-based application orchestration platform. It has a number of useful features, including automated application deployment, resource management, scaling, and load balancing [3]. Docker and Kubernetes work best together to organize containerized applications. [4].

The Internet of Things (IoT), which includes large structures such as smart cars, smart homes, and smart cities,

is posing a challenge to the traditional approach to service deployment in cloud data centers. When traditional distributed data center approaches are used for IoT, the core network is burdened by the transmission of mostly unnecessary data. Yi et al. propose Fog and Edge Computing as one way to move virtualized cloud data center services to the network edge. Recent advances in the design of energy-efficient, cheap, and low-cost single board computers have made it simple to place computing power close to service consumers [5].

A Kubernetes operator may run into a number of resource-related issues, such as the cluster's resource utilization is approaching critical limits, the desired number of pods are not running, and failures or misconfiguration prevent pods or nodes from participating in the cluster. Having a real-time view of resource usage and availability for individual nodes or pods can help avoid such issues.

The aim of this paper is to describe the design and operation of Kube-Monitor, a real-time monitoring framework that is used to dynamically collect metrics from a heterogeneous cluster comprised of traditional PC and Raspberry Pi devices. Kube-Monitor will rely on a number of tools to collect and update 15 different cluster resource metrics every five seconds.

II. LITERATURE SURVEY

Yang et al.[1] [(2017) presented a kubernetes monitoring framework that provides a comprehensive monitoring mechanism that integrates and provides relatively complete system resource utilization and application QoS metrics.

Grohmann et al. [5] (2017) developed "PyMon" to monitor single board computers (SBCs). PyMon is a monitoring data collection tool based on the most recent version of "monit" and a "Django" application. It's delivered via a number of Docker images and allows for monitoring with a low processing overhead.

Muralidharan et al. [6] (2019) presented a container-based IoT application for efficient monitoring and management in a smart city scenario. They used Heapster to enable monitoring of the IoT applications clustered under their platform in Kubernetes, which can collect resource utilization information from each node and display it in the Kubernetes dashboard.

Shemakhinskaya [7] (2020) proposed using the Kubernetes "operator" for a tool that can efficiently obtain information about the state of hard drives in the system to automate the collection of hard drive metrics.

Carcassi et al. [8] (2020) presented "SLATE" as Distributed Kubernetes Clusters Monitoring. Prometheus, Thanos, Grafana were chosen because they are specifically designed to handle time series over long time intervals combined from multiple data sources.

III.BACKGROUND

This section introduces several components that are approaching in this paper: Edge Computing, Raspberry Pi, Docker, and Kubernetes.

A. *Edge Computing* : In the post-Cloud computing era, the Internet of Things (IoT) proliferation and widespread adoption of 4G/5G have resulted in the emergence of a new computing paradigm known as edge computing, in which data is processed at the network's edge [9]. Edge computing analyzes data by utilizing the compute power of resources that are not located in traditional data centers. These computational resources are close to the data sources. This type of computing provides services such as data processing, data caching/storage, service delivery, privacy protection and IoT management [10].

B. *Raspberry Pi* : It's commonly referred to as a Single Board Computer, which means it runs a full operating system and has enough peripherals (memory, CPU, and power regulation) to begin execution without additional hardware. The Raspberry Pi can run a variety of operating systems and only needs power to start up [11]. Many of these boards can be linked together to form small, low-cost clusters that mimic the features of large data centers, enabling new Fog and Edge Compute applications in which computation is pushed out from the network's core towards data sources. This can help improve privacy and reduce bandwidth requirements while also decentralizing the architecture, but it comes at the cost of increased management complexity [12].

C. *Docker* : Solomon Hykes, the founder of dotCloud, first introduced Docker at the Python Developer's Conference in California on March 15, 2013. Docker is an open-source platform for developing, deploying, and running containerized applications. The applications can be separated from the infrastructure with Docker. Docker makes use of cgroups and namespaces, two of the most important kernel features [13]. A container contains everything an application needs to run, such as system tools, libraries, and resources, while isolating it from the infrastructure on which it is running, forming a kind of virtualisation layer [14]. The Docker model is based on a client-server architecture. The core components of the Docker architecture are as follows [13]:

- Docker daemon: Docker daemon's main function is to manage containers running on a host system.
- Docker client: is a command-line interface that allows users to interact with the Docker daemon.
- Docker images: are the fundamental components of Docker. Docker launches containers from these images, so they're considered source code for running containers.

- Docker registries: Images are stored in registries. They come in two varieties: public and private.
 - Container : a runnable instance of a Docker image is referred to as a container.

Figure 1 shows the Docker architecture.

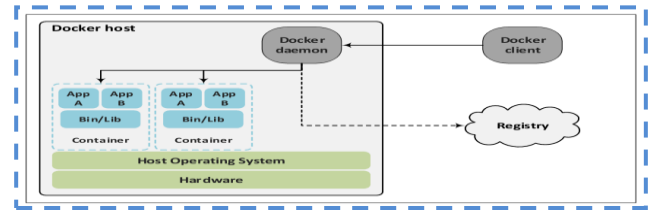


Fig.1. Docker Engine Architecture[13]

D. *Kubernetes* : Kubernetes is a Google-developed system for managing containerized applications in a clustered environment [4]. Kubernetes is widely used in industry, with 59 percent of large organizations using it in production [15]. It automates container-based application deployment, sizing, and maintenance. It is based on the pods principle, with each pod consisting of one or more containers that have been co-located, co-scheduled, and run in the same manner [16]. A local daemon (the Kubelet) manages the lifecycle of pods assigned to worker nodes [15]. The typical Kubernetes architecture consists of two types of computing nodes which allow the system to function. The master node (aka the control plane) and a set of worker nodes [17]. Fig. 2 shows Kubernetes Architecture.

The Control Plane is made up of the following components [17][18][19]:

- API server (kube-apiserver) : is the control plane's frontend, where API requests are processed.
- Etcd : is the store where Kubernetes maintains all of its data, such as what nodes exist, what resources exist on the cluster, and so on.
- Scheduler (kube-scheduler) : The job of the kube-scheduler is to find a computing node for a pod to run on.
- Controller Manager: comprises multiple controllers that perform cluster-level functions like replicating components, tracking worker nodes, and handling node failures, and so on .

Each worker node contains the following components[18][19]:

- Kubelet: drives the container runtime to start running pods that a scheduler has assigned to its node and monitoring their status.
- kube-proxy: represents the core networking component of K8s and is responsible for routing requests between Pods on different nodes, and between Pods and the internet.
- Container runtime : is responsible for starting and stopping the containers that exist in each pod. Kubernetes supports several container runtimes, such as Docker ,Containerd, rkt and CRI-O.

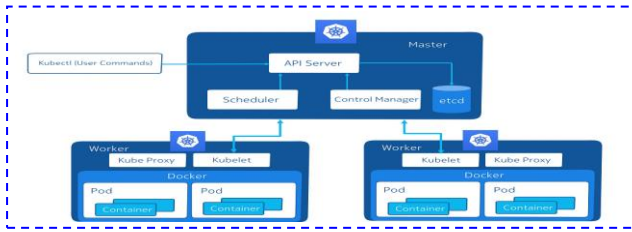


Fig.2. Kubernetes Architecture [20]

IV.KUBE-MONITOR FRAMEWORK COMPONENTS

The established real-time monitoring system is based on the use of several tools that depend on each other to provide real-time resources monitoring. The following are the tools that were used:

A.Advisor (Container Advisor): is an open-source agent integrated into the kubelet binary that monitors resource usage and collects metrics from the containers and nodes on which it is installed.

B.Kubelet: is a critical agent that runs on every node in the cluster. It is the component responsible for ensuring that the containers are running in the nodes. Kubelet operates declaratively, receiving PodSpecs and ensuring that the current state corresponds to the desired pods.

C. Metrics-server: metrics-server considers as a resource usage data aggregator for the entire cluster. It collects data about resource usage from Kubelet. It is registered to the main API server and exposes metrics through the Metrics API. Metrics-server is expected to be included in modern Kubernetes distributions, but it is currently available as an add-on component. Each cluster will only have one instance of the metrics server running.

D. API-server : The Kubernetes control plane's front end is represented by the API server.

E.K8s_Metrics_Collector: This component represents the module that is programmed using Go language in this work to collect the required real-time resource monitoring metrics data.

F.Prometheus : In most cases, metrics are available via the HTTP server's /metrics endpoint, so metrics scraper tools are required in a production environment to scrape and store metrics from various sources. Prometheus Server is configured in the established monitoring environment to collect metrics from K8s_Metrics_Collector periodically and store them in a time-series database.

G. Grafana : It is a monitoring tool that pulls data from Prometheus and displays it in visually appealing dashboards.

V.KUBE-MONITOR STRATEGY

The established monitoring system collects real-time resource metrics in the following manner:

Step 1: cAdvisor collects, aggregates and exports metrics about containers and nodes that on which it is deployed.

Step 2: Kubelet retrieves metrics from cAdvisor through Kubelet APIs, aggregate them, and expose them for consumption by Metrics-server.

Step 3: Metrics-Server locates all available nodes and queries the Kubelet API for information on container and

node usage. Thus, Metrics-Server will periodically retrieve metrics from Kubeletes running on all the nodes .

Step 4 : API Server which has the Master Metrics API will provide access to Metrics-server data.

Step 5 : The K8s_Metrics_Collector requests data from the Metrics Server via the API Server, which is equipped with the Master Metrics API.

Step 6 : Prometheus pulls metrics directly from K8s_Metrics_Collector .

Step 7 : After building a specialized Grafana dashboard for K8s_Metrics_Collector on top of Kubernetes cluster monitoring , the dashboard visualizes the collected metrics.

Figure 3 depicts Kube-Monitor which represents the Established Kubernetes Monitoring Framework Architecture.

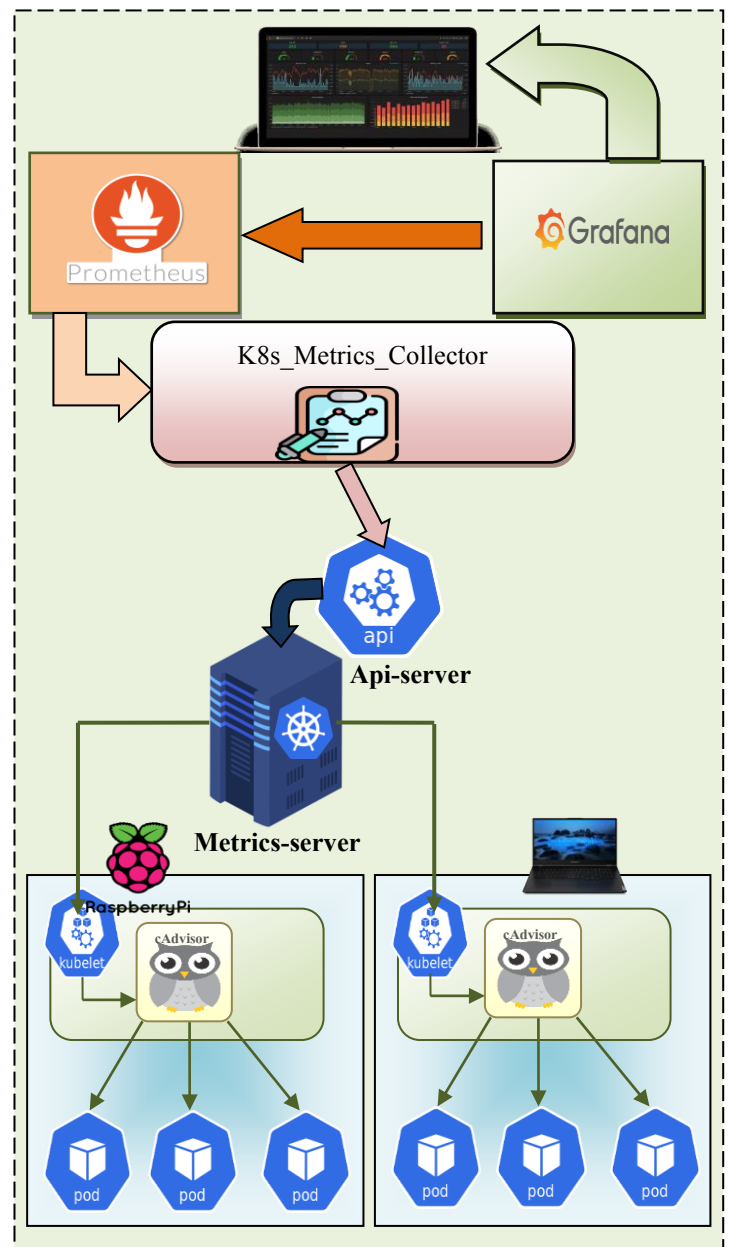


Fig.3. Kube-Monitor Architecture

VI. KUBE-MONITOR METRICS

Metrics collected by the Kube-Monitor can be divided into two categories:

- A. *Node-Level* Resource Metrics Which includes the following metrics:
 - Allocatable CPU Cores: is the number of CPU cores in each node that are available to pods.
 - Allocatable memory bytes: is the number of memory bytes in each node that are available to pods.
 - Limits CPU Cores: Quantity of CPU max (limit) requestable by the pods in their pod's manifest file.
 - Limits memory bytes: Quantity of memory max (limit) requestable by the pods in their pod's manifest file.
 - Requests CPU Cores: The quantity of CPU requested by pods, as specified in their manifest.
 - Request memory bytes: The amount of memory requested by pods, as specified in their manifest.
 - Usage CPU Cores: A node's total CPU usage.
 - Usage memory bytes: A node's total memory usage.
 - Pod counts: which correspond to the Pods found in each node.
- B. *Pod-Level* Resource Metrics Which includes the following metrics:
 - Pod Request CPU Cores: It's the minimum amount of CPU cores required to run a pod.
 - Pod Request memory bytes: It's the minimum amount of memory bytes required to run a pod.
 - Pod limits CPU Cores: The maximum amount of CPU cores that the node will allow the pod to use.
 - Pod limits memory bytes: The maximum amount of memory bytes that the node will allow the pod to use.
 - Pod Usage CPU cores: Total CPU in use by a pod
 - Pod Usage memory bytes: Total memory bytes in use by a pod.

VII. EXPERIMENTAL CLUSTER

The testing cluster where tests have been deployed and executed, is a bare metal cluster made up of four nodes: one master and three worker nodes, each one deployed in a different physical machine.

The master is deployed in a Lenovo Legion Y530-15ICH laptop with 8th Generation Intel® Core™ i5-8300H Quad Core Processor, 16 GB DDR4 RAM, 1 TB disk capacity. The OS kernel is 5.11.0-40-generic with Ubuntu 20.04.3 LTS (Focal Fossa) Distro and the OS type is 64 bit.

As a laptop Worker node, an HP specter x360 with an Intel® Core™ i7-8565U CPU, 16 GB memory, and 512 GB SSD storage was used. The OS kernel is 5.11.0-40-generic with Ubuntu 20.04.3 LTS (Focal Fossa) Distro and the OS type is 64 bits.

As edge worker nodes, two Raspberry Pi 4 Model B computers are used. Each Raspberry Pi has the following characteristics: 64-bit quad-core Cortex-A72 processor, 8GB LPDDR4 RAM. The OS is ubuntu 21.04 (Hirsute Hippo Distro).

HIKVISION Ethernet switch model DS -3E0105P-E/M with 5 ports has been used to connect the nodes.

Table I summarizes the cluster tools specification.

TABLE I: CLUSTER TOOLS SPECIFICATION

Tool	Specification
Docker	20.10.7
Kubernetes	V1.23.1
Metrics-Server	V0.4.5
Prometheues	2.23.1
Grafana	V8.3.x
Go Lang	go1.17.5
Kube-Monitor programming Language	Go
Manifest Language	YAML

VIII. EXPERIMENTAL RESULTS

The experimental results showed that Kube-monitor provides a comprehensive overview of two types of metrics: node-level metrics and pod-level metrics. As node-level metrics the following metrics are presented: Allocatable CPU Cores, Allocatable memory bytes, Limits CPU Cores, Limits memory bytes, Requests CPU Cores, Request memory bytes, Usage CPU Cores, Usage memory bytes, and pod counts Fig.4 illustrates the built Grafana representation of the node-level metrics.

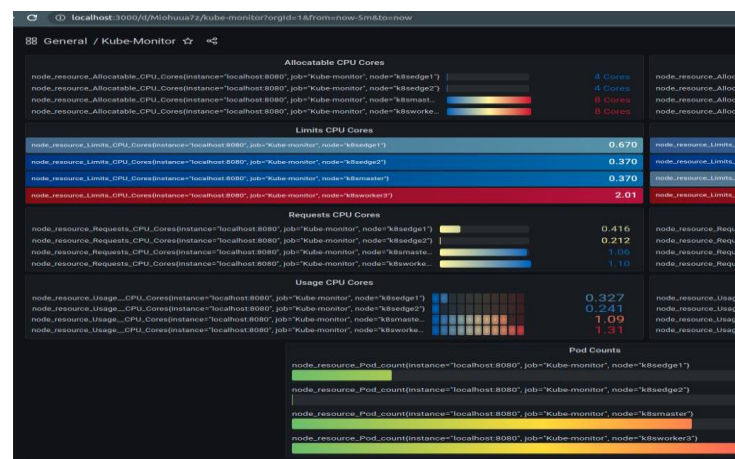


Fig.4. Kube-Monitor Metrics at the Node Level

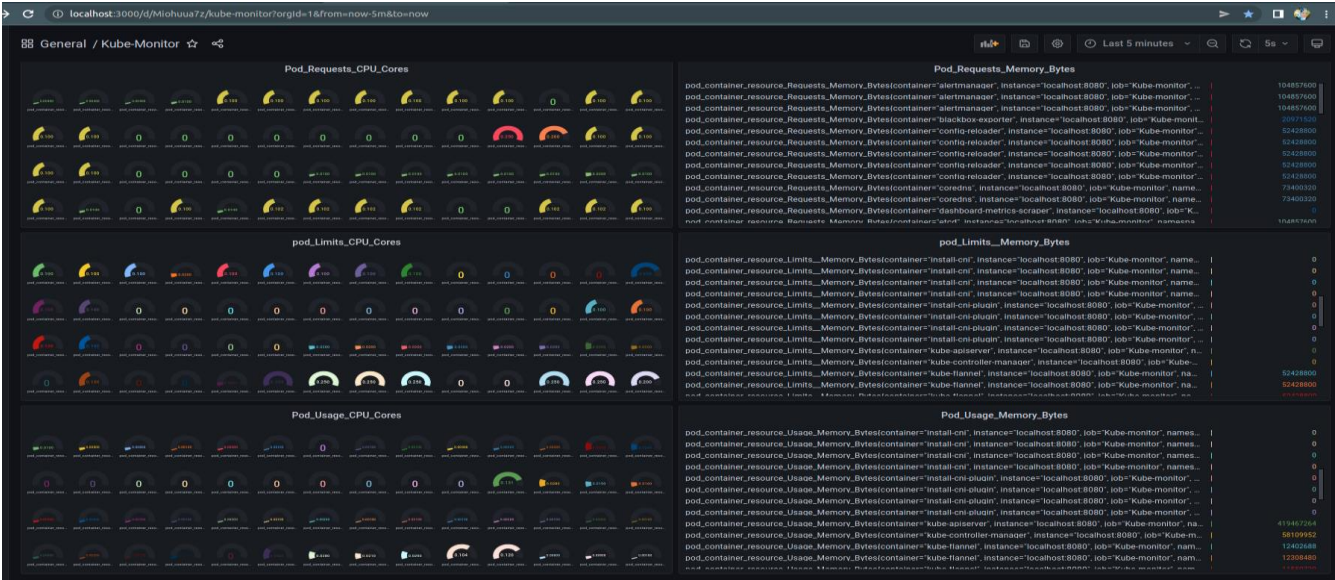


Fig. 5. Kube-Monitor Metrics at the Pod Level

As pod-level metrics the following metrics are presented: Pod Request CPU Cores, Pod Request memory bytes, Pod limits CPU Cores, Pod limits memory bytes, Pod Usage CPU cores, and Pod Usage memory bytes. Fig. 5 illustrates the built Grafana representation of the pod-level metrics.

IX.CONCLUSIONS AND FUTURE WORK

The Kube-Monitor framework was introduced in this paper as a generic framework for resource monitoring in a heterogeneous Kubernetes cluster made up of laptop devices and modern Raspberry Pi devices. Kube-Monitor considers both system and application resource metrics. Kube-Monitor will depend on the integration of several tools to dynamically collect and update 15 different cluster resource metrics every five seconds. As a result, the Kubernetes operator can see a fairly comprehensive set of monitored metrics. In the future, Kube-monitor will be integrated with Kubernetes scheduler, allowing scheduling decisions to be made based on aggregated real-time resource metrics.

REFERENCES

- [1] C. C. Chang, S. R. Yang, E. H. Yeh, P. Lin, and J. Y. Jeng, "A Kubernetes-Based Monitoring Platform for Dynamic Cloud Resource Provisioning," *2017 IEEE Glob. Commun. Conf. GLOBECOM 2017 - Proc.*, vol. 2018-January, pp. 1–6, 2017, doi: 10.1109/GLOCOM.2017.8254046.
- [2] Arnav Bhutani, "Building and Evaluating a Prototype Edge Computing System," Oregon State University, 2021.
- [3] N. D. Nguyen and T. Kim, "Balanced leader distribution algorithm in kubernetes clusters," *Sensors (Switzerland)*, vol. 21, no. 3, pp. 1–15, 2021, doi: 10.3390/s21030869.
- [4] E. Kristiani, C. T. Yang, C. Y. Huang, Y. T. Wang, and P. C. Ko, "The Implementation of a Cloud-Edge Computing Architecture Using OpenStack and Kubernetes for Air Quality Monitoring Application," *Mob. Networks Appl.*, vol. 26, no. 3, pp. 1070–1092, 2021, doi: 10.1007/s11036-020-01620-5.
- [5] M. Großmann and C. Klug, "Monitoring Container Services at the Network Edge," *Proc. 29th Int. Teletraffic Congr. ITC 2017*, vol. 1, pp. 130–133, 2017, doi: 10.23919/ITC.2017.8064348.
- [6] S. Muralidharan, "Monitoring and Managing IoT Applications in Smart Cities Using Kubernetes," *CLOUD Comput. 2019 Tenth Int. Conf. Cloud Comput. GRIDS, Virtualization Monit.*, no. Venice, Italy, pp. 1–6, 2019.
- [7] A. S. Shemykinskaya, "Hard drives monitoring automation approach for Kubernetes container orchestration system," *Proc. ISP RAS, 2020, Volume 32, Issue 2*, pp. 99–106, 2020.
- [8] G. Carcassi, J. Breen, L. Bryant, R. W. Gardner, S. McKee, and C. Weaver, "SLATE: Monitoring Distributed Kubernetes Clusters," *ACM Int. Conf. Proceeding Ser.*, pp. 19–25, 2020, doi: 10.1145/3311790.3401777.
- [9] F. Liu, G. Tang, Y. Li, Z. Cai, X. Zhang, and T. Zhou, "A Survey on Edge Computing Systems and Tools," *Proc. IEEE*, pp. 1–24, 2019, doi: 10.1109/JPROC.2019.2920341.
- [10] ÖFIT, *Edge Computing - Edge Computing*, vol. 08. CRC Press is an imprint of Taylor & Francis Group, LLC, 2020.
- [11] S. J. Johnston and S. J. Cox, "The raspberry Pi: A technology disrupter, and the enabler of dreams," *Electron.*, vol. 6, no. 3, 2017, doi: 10.3390/electronics6030051.
- [12] S. J. Johnston *et al.*, "Commodity single board computer clusters and their applications," *Futur. Gener. Comput. Syst.*, vol. 89, pp. 201–212, 2018, doi: 10.1016/j.future.2018.06.048.
- [13] A. Javed and A. K. Heljanko, "Container-based IoT Sensor Node on Raspberry Pi and the Kubernetes Cluster Framework," 2016.
- [14] R. Scolati, I. Fronza, N. El Ioini, A. Samir, and C. Pahl, "A

- containerized big data streaming architecture for edge cloud computing on clustered single-board devices,” *CLOSER 2019 - Proc. 9th Int. Conf. Cloud Comput. Serv. Sci.*, no. Closer, pp. 68–80, 2019, doi: 10.5220/0007695000680080.
- [15] A. Jeffery, H. Howard, and R. Mortier, “Rearchitecting Kubernetes for the Edge,” *EdgeSys 2021 - Proc. 4th Int. Work. Edge Syst. Anal. Networking, Part EuroSys 2021*, pp. 7–12, 2021, doi: 10.1145/3434770.3459730.
- [16] D. Jorge-Martinez *et al.*, “Artificial intelligence-based Kubernetes container for scheduling nodes of energy composition,” *Int. J. Syst. Assur. Eng. Manag.*, no. October, 2021, doi: 10.1007/s13198-021-01195-8.
- [17] M. Luksa, *Marko Lukša*. 2018.
- [18] J. Arundel and J. Domingus, *Cloud Native DevOps with Kubernetes: Building, Deploying, and Scaling Modern Applications in the Cloud*. O’Reilly Media, Inc., 2019.
- [19] J. K. B. T. Michael Elder, *hybrid cloud apps with openshift and kubernetes*. O’Reilly Media, Inc., 2021.
- [20] M. Tavakkoli, “Analyzing the Applicability of Kubernetes for the Deployment of an IoT Publish/Subscribe System,” Faculty of Electrical Engineering, Mathematics, and Computer Science Delft University of Technology, 2019.